

TABLA DE CONTENIDO

NIVEL FUNDAMENTAL	1
MÓDULO 2. FUNDAMENTOS DEL LENGUAJE	1
Conceptos básicos	1
Tipos básicos y colecciones de Python	2
Control de flujo	3
Errores y Excepciones	4
Expresiones regulares	6
Archivos JSON	8

NIVEL FUNDAMENTAL

MÓDULO 2. FUNDAMENTOS DEL LENGUAJE

CONCEPTOS BÁSICOS

Sintaxis. En Python, a diferencia de Java, el código no lleva un signo de terminación por línea.

Indentación. Las llaves de Java aquí son sustituidas por “:” para iniciar el bloque, e indentación para indicar qué instrucciones le pertenecen.

Variables. Python determina automáticamente el tipo del valor. Así mismo, pueden sobrescribirse posteriormente porque utiliza tipado dinámico.

Reglas básicas para nombres. Un nombre puede contener letras, números (solo no al inicio) y guiones bajos

Nombres válidos	Nombres inválidos
nombre_usuario	nombre-usuario
edad7	7edad
_total	nombre_Usuario
	(Nombres de funciones o palabras reservadas)

Alcance de las variables. De acuerdo con las siguientes definiciones, se recomienda limitar el uso de las *variables globales*.

- ❖ **Alcance local.** Una variable creada dentro de una función *solo existe* dentro de ella.
- ❖ **Alcance global.** Una variable definida fuera de una función puede ser leída en *todo* el código, incluyendo dentro de las funciones.

TIPOS BÁSICOS Y COLECCIONES DE PYTHON

Tipos básicos	Ejemplo
<i>Int</i>	<code>edad = 30</code>
<i>Float</i>	<code>precio = 19.99</code>
<i>Str</i>	<code>nombre = "Ana"</code>
<i>Bool</i>	<code>activo = True</code>
<i>Ausencia de valor</i>	<code>resultado = None</code>

El tipo de cada variable se puede consultar con `print(type(edad))`.

Conversión de Tipos. Es posible convertir valores, definiendo la variable con el tipo básico y entre paréntesis el valor, cuidando la compatibilidad del tipo con el argumento.

```
numero = int ("10")
texto = str(25)
```

Listas (*list*). Son ordenadas y modificables. Se escriben entre **corchetes**.

```
colores = ["rojo", "verde", "azul"]
```

Acceso por índice:

```
print(colores[0])
```

Modificar:

```
colores.append("amarillo")
colores[1] = "verde oscuro"
```

Las listas pueden contener **distintos tipos**, aunque normalmente conviene mantener datos relacionados:

```
datos = ["Ana", 30, True]
```

Tuplas (*tuple*). Se utilizan para representar que no deberían cambiar. Similar a las listas, es ordenada, más no modificable. Se escriben entre **paréntesis**.

```
coordenadas = (10, 20)
```

Puedes acceder a sus elementos utilizando **print**:

```
print(coordenadas[0])
```

Diccionarios (*dict*). Almacena pares clave-valor, y permite acceder, modificar o agregar contenido.

```
usuario = {
    "nombre": "Ana",
    "edad": 30,
    "activo": True,
}
```

Acceder:

```
print(usuario["nombre"])
```

Modificar:

```
usuario["edad"] = 31
```

Agregar:

```
usuario["ciudad"] = "Madrid"
```

Para evitar un error si la clave no existe:

```
print(usuario.get("telefono"))
```

Conjuntos (set). Almacena valores únicos y no depende de un orden fijo. No permite valores repetidos, por lo que son útiles para eliminar duplicados y realizar operaciones como **unión** o **intersección**.

```
roles = {"admin", "editor", "admin"}
print(roles)
```

El resultado solo contiene una vez "admin".

Agregar y eliminar:

```
roles.add("lector")
roles.remove("editor")
```

Resumen

Tipo	Ordenado	Modificable	Permite duplicados
<i>list</i>	Sí	Sí	Sí
<i>tuple</i>	Sí	No	Sí
<i>dict</i>	Sí, por inserción	Sí	Claves únicas
<i>set</i>	No debe asumirse	Sí	No

CONTROL DE FLUJO

Controles de Flujo	Ejemplo 1	Ejemplo 2
Condicionales. Bloques separados por ":" e indentación.	<pre>if edad >= 18: print("Es mayor de edad") elif edad >= 13: print("Es adolescente") else: print("Es menor de edad")</pre>	
Bucle (for). Para recorrer elementos, como de una lista o diccionario.	<pre>colores = ["rojo", "verde", "azul"] for color in colores: print(color)</pre>	<pre>usuario = {"nombre": "Ana", "edad": 30} for clave, valor in usuario.items(): print(clave, valor)</pre>
Bucle (while). Se ejecuta mientras una condición	<pre>contador = 0 while contador < 3:</pre>	

sea verdadera. Evitar bucles infinitos.	<pre>print(contador) contador += 1</pre>	
Break y Continue. <i>Break</i> termina un bucle, mientras que <i>continue</i> salta la iteración actual.	<pre>for numero in range(10): if numero == 3: break print(numero)</pre>	<pre>for numero in range(5): if numero == 2: continue print(numero)</pre>
Match y Case. En Python no hay un switch como en Java, por eso se usa <i>Match</i> y <i>Case</i> , donde puede escribirse “_” como alternativa predeterminada en un case.	<pre>opcion = 2 match opcion: case 1: print("Crear") case 2: print("Consultar") case 3: print("Salir") case _: print("Opción no válida")</pre>	
Pattern matching. <i>match</i> también puede comparar estructuras con determinadas claves y asigna el valor de <i>codigo</i> .	<pre>respuesta = {"estado": "ok", "codigo": 200} match respuesta: case {"estado": "ok", "codigo": codigo}: print(f"Solicitud correcta: {codigo}") case {"estado": "error", "codigo": codigo}: print(f"Solicitud fallida: {codigo}") case _: print("Respuesta desconocida")</pre>	

El uso de *match* puede ser útil cuando:

- Hay varias alternativas.
- Se comparan estructuras de datos.
- Se desea reducir una cadena extensa de *if*.
- Se necesita descomponer listas, tuplas o diccionarios.

ERRORES Y EXCEPCIONES

Errores y Excepciones	Ejemplo 1	Ejemplo 2
Try y Except. Se utiliza <i>try</i> para colocar la operación que podría fallar y <i>except</i> para manejar una excepción. Así el programa no termina abruptamente.	<pre>try: numero = int("abc") except ValueError: print("El valor no es un número válido")</pre>	
Excepciones específicas. Es preferible indicar que	<pre>try: numero = int(input("Número: "))</pre>	

excepción esperamos para conocer el problema y facilitar el diagnóstico. Se pueden manejar distintas excepciones por separado.

```
resultado = 10 / numero
except ValueError:
    print("Debes escribir un número")
except ZeroDivisionError:
    print("El número no puede ser cero")
```

Else y Finally. *else* solo se ejecuta si no ocurrió ninguna excepción, y *finally* se ejecuta siempre.

```
try:
    numero = int("10")
except ValueError:
    print("Valor inválido")
else:
    print(f"Número correcto: {numero}")

try:
    archivo = open("datos.txt")
except FileNotFoundError:
    print("El archivo no existe")
finally:
    print("Proceso finalizado")
```

Excepción intencional. Se puede generar una excepción con *raise*, lo que permite validar condiciones. Adicionalmente, interrumpe la ejecución y comunica formalmente que ocurrió un error.

```
edad = -1

if edad < 0:
    raise ValueError("La edad no puede ser negativa")
```

Manejo de excepciones de raise. Raise interrumpe la ejecución, pero si es atrapada por try-except, el programa puede continuar.

```
def validar_precio(precio):
    if precio < 0:
        raise ValueError("El precio no puede ser negativo")

    print("Precio válido")
[...]
```

```
try:
    validar_precio(-5)
except ValueError as error:
    print(f"Error: {error}")
```

Recomendaciones para el Control y Gestión de Excepciones

- No ocultar errores en las excepciones usando simplemente Exception; es recomendable registrar y explicar el error.
- Es mejor validar antes de procesar. Por ejemplo, dentro de una función primero usar try-except para validar y después procesar los datos.
- Comunicar errores mediante raise.
- Crear excepciones propias mediante clases. Esto hace que el código sea más expresivo.
- Registrar errores mediante el paquete logging, útil en aplicaciones reales.

Adicionalmente, las excepciones se pueden guardar en una variable “error” (por convención), lo que permite consultar la información guardada del error. Entonces se recomienda para:

- Mostrar el mensaje original.
- Registrar detalles con logging.
- Analizar la excepción.
- **Encadenarla** con otra excepción.
- Incluir información adicional en un mensaje.

A su vez, un **raise** contenido en una excepción puede llevar “**from error**”, para adjuntar el nuevo mensaje de raise, sin borrar la causa original definida por **except**.

- Raise – Lanza una excepción e interrumpe el flujo normal.
- As error – Guarda la excepción de una variable.
- From error – Conserva la excepción original como causa de una nueva.

Ejemplo:

Código	Resultado de la excepción
<pre>try: precio = float(valor) except ValueError: raise ValueError("El precio debe ser numérico")</pre>	ValueError: El precio debe ser numérico
<pre>try: precio = float(valor) except ValueError as error: raise ValueError("El precio debe ser numérico") from error</pre>	ValueError: could not convert string to float: 'abc' The above exception was the direct cause of the following exception: ValueError: El precio debe ser numérico

EXPRESIONES REGULARES

Expresión regular. Patrón utilizado para buscar, validar o extraer texto. No es recomendable para patrones complejos o si se desea analizar estructuras como JSON completo (para ello, usar el módulo estándar **json**).

- ❖ Una expresión regular no es adecuada para interpretar JSON completo porque puede contener objetos anidados, listas dentro de objetos, valores con comas, llaves y corchetes, reglas de escape; entre otras cosas.

Módulo estándar: “re”

Usos de re	Descripción	Ejemplo
-------------------	-------------	---------

re.search	Buscar coincidencias. La 'r' antes de la cadena indica una cadena "raw", donde las barras invertidas se interpretan correctamente. Es un <i>booleano</i>.	<pre> texto = "Código: 12345" resultado = re.search(r"\d+", texto) if resultado: print(resultado.group()) </pre>
re.fullmatch	Comprobar que todo el código cumple el patrón. Si cumple, devuelve un objeto de coincidencia, si no, devuelve <i>none</i>. Del ejemplo: [A-Z]: una letra mayúscula. {3}: exactamente tres repeticiones. -: un guion literal. \d: un dígito. {3}: exactamente tres dígitos.	<pre> codigo = "ABC-123" if re.fullmatch(r"[A-Z]{3}-\d{3}", codigo): print("Código válido") else: print("Código inválido") </pre>
re.findall	Devuelve todas las coincidencias en formato de lista, cuando no se utiliza grupos de captura.	<pre> texto = "Pedidos: 120, 350 y 480" numeros = re.findall(r"\d+", texto) print(numeros) </pre>
.group	Los paréntesis permiten extraer partes específicas del objeto de coincidencia. En el ejemplo, de "resultado".	<pre> correo = "ana@example.com" resultado = re.fullmatch(r"([\w.-]+)@([\w.-]+)", correo) if resultado: usuario = resultado.group(1) dominio = resultado.group(2) print(usuario) print(dominio) </pre>

Para complementar lo anterior, los patrones básicos para **cadenas raw 'r'** son:

Patrón	Significado
\d	Un dígito
\w	Letra, número o guion bajo
\s	Espacio en blanco
.	Cualquier carácter
^	Inicio del texto
\$	Final del texto
+	Una o más repeticiones
*	Cero o más repeticiones
?	Cero o una repetición

ARCHIVOS JSON

En Python se utiliza el módulo estándar *json* para interactuar con archivos JSON. Para manejar errores de archivo y formato se usan las *excepciones*:

- ❖ `FileNotFoundError` – Si el archivo no existe.
- ❖ `json.JSONDecodeError` – Si el archivo no contiene JSON válido.